总览：综合分析

Claude Code 工程系统综合分析报告

一、系统定位：LLM Agent 的工程化操作系统

Claude Code 并非简单的"LLM + 工具"封装，而是一套**以对话为内核、以工具为四肢、以分层压缩为代谢机制、以文件系统为持久化记忆层、以 Hook 为扩展神经的上下文感知型 Agent 运行时**。

核心矛盾：**无限复杂的真实世界（代码库、对话历史、跨会话上下文）与有限上下文窗口之间的永恒张力。** 所有子系统——上下文注入、记忆持久化、工具编排、可靠性——都是这一矛盾的不同解法。

二、核心架构总览

```
用户输入
    │
    ▼
processUserInput（意图识别管道）
    │ 贪心字符串匹配 → Slash / Bash / Ultraplan / LLM Prompt
    ▼
命令队列（优先级: now / next / later）
    │
    ▼
QueryEngine（会话生命周期管理者）
    │
    ▼
query() 核心循环（AsyncGenerator 状态机）
    │
    ├── 上下文压缩（snip → microcompact → contextCollapse → autocompact）
    ├── callModel() 流式 API 调用
    ├── 工具识别（tool_use block）
    │
    ▼
工具编排层（StreamingToolExecutor）
    │
    ├── 分类: isConcurrencySafe 分区
    ├── 执行: runToolUse 8 阶段管线
    └── 结果聚合 → yield 给调用方
    │
    ▼
Hook 系统（22 个生命周期检查点）
    │
    ▼
循环直到 needsFollowUp === false
```

三、意图识别与路由：规则优先，LLM 兜底

3.1 三层分流策略

层级	策略	依据
第一层	字符串前缀匹配（/、!）	CLI 输入具有天然结构性，零 LLM 开销
第二层	关键词检测（ultraplan）	Feature flag 暗启动态路由
第三层	LLM 语义判断	仅在规则无法处理时介入

通俗解释：用哨兵检测枪声（字符串规则），而非每次都派侦察兵去确认（LLM 语义分类）。

3.2 统一命令模型

所有可执行单元（内置命令、Skill、斜杠命令、MCP 工具）共享同一 `Command` 接口，通过 `type` 字段区分执行路径：

- **`prompt` + `inline`** → 读取 SKILL.md，参数替换，加入消息历史
- **`prompt` + `fork`** → Fork 子 Agent 异步执行，结果返回主 Agent
- **`local`** → 懒加载模块，同步执行，零 API token 消耗
- **`local-jsx`** → 渲染 React/Ink UI 组件

3.3 统一命令队列优先级

所有命令（用户输入、任务通知、权限请求）共享同一队列，三级优先级保证公平性：

- `now (0)`: 立即执行，中断当前操作
- `next (1)`: 下一轮，用户输入默认级别
- `later (2)`: 最后执行，后台任务通知

核心不变量：用户输入永远不被系统通知饿死。

四、上下文管理：六层压缩体系

4.1 层级总览

```
Layer 5: 持久化记忆提取（ExtractMemories）→ 跨会话
Layer 4: 会话记忆（Session Memory）→ 当前会话增量笔记
Layer 3: 全量压缩（Compaction）→ AI 摘要
Layer 2: Snip 历史裁剪（默认关闭）
Layer 1: 微型压缩（Microcompact）→ 工具结果清理
```

```
Layer 0: Token 计数与阈值检测
```

通俗类比：Token 是有限 RAM，Microcompact 是即时垃圾回收，Compaction 是定期深度整理，Session Memory 是增量备份笔记，ExtractMemories 是定期归档到硬盘。

4.2 Microcompact 的双重缓存优化

Microcompact 有两种触发模式，分别对应 API 缓存的不同机制：

时间触发型：Prompt Cache 有 1 小时 TTL。距上次 Assistant 消息超过 60 分钟后，缓存已自然过期，此时清理旧工具结果可减少重新序列化数据量，但不破坏任何缓存命中机会。

缓存编辑型（Cached Microcompact）：API 的 `cache_edits` 机制允许在已缓存前缀上声明"删除某个段落"，服务器端将对应 KV cache 标记为失效，但**其余缓存仍然有效**。Claude Code 利用这一能力发送 `cache_edits` 删除旧工具结果——旧结果从上下文消失，但 KV 缓存不需要重建，**双赢操作**。

4.3 Compaction 的强制结构化摘要

全量压缩的核心不是"删除旧消息"，而是"用 AI 生成能替代旧消息的结构化摘要"。摘要提示词强制要求 9 个结构化部分：用户意图、技术概念、文件操作、错误修复、待完成任务、当前进展等。

设计动机：LLM 在长对话中存在注意力丢失问题，自然语言摘要容易遗漏关键实体（文件名、函数签名）。强制结构化格式迫使模型必须明确写出关键实体名，确保压缩后的上下文在关键信息维度上不降级。

Fork 子 Agent 优先路径：压缩通过 `runForkedAgent` 执行，继承父会话的 `cacheSafeParams`，使压缩请求的 API cache key 与父会话完全相同。

4.4 Session Memory Compact：颠覆性设计

传统压缩每次都要调用 AI 生成摘要。Session Memory Compact 提前用子 Agent 增量维护一份 Markdown 笔记，压缩时直接读文件，不需要 API 调用——**零成本压缩**。系统优先尝试 SM-Compact，失败才回退到传统压缩。

4.5 Token 计数盲区

Claude API 的 streaming 模式为每个 content block 发射独立的 assistant 消息记录（共享 `message.id`），在并行 tool calls 之间插入 tool_result。如果统计逻辑只抓取最后那条 assistant 记录，就会漏掉中间插入的巨大 tool_result 文本，导致严重低估。Claude Code 的回溯逻辑正是为了堵住这个盲区。

五、记忆系统：持久化上下文的范式设计

5.1 四层记忆类型

类型	作用域	典型内容	保存规范
user	私有（绝不升）	用户角色、偏好、工作风格	"什么"而非"什么发生了"
feedback	私有（默认）	协作偏好、反馈教训	成功与失败同等重要
project	团队（强烈偏向）	项目上下文、团队约定	可推导的不保存
reference	团队（通常）	外部系统指针（Linear/Grafana）	需时效性警告

5.2 可推导信息不保存原则

铁律：可以直接捡回来的信息不保存。具体排除：

- 代码模式、架构（可从项目当前状态推导）
- Git 历史（`git log` / `git blame` 是权威）
- 调试解决方案（修复代码本身才是权威）
- CLAUDE.md 中已有的内容
- 临时任务细节

即使用户主动要求保存 PR 活动日志，系统也会要求追问"背后的洞见是什么"——**记忆的价值不在于记录"发生了什么"，而在于凝固"学到了什么"**。

5.3 extractMemories 的协作互斥

核心设计：**不是两套系统竞争，而是"主 Agent 先写，Forked Agent 补漏"**：

- 主 Agent 主动写了记忆 → Forked Agent 检测到后跳过，推进游标
- 主 Agent 没有写记忆 → Forked Agent 启动，做无差别结构化提取

主 Agent 写的是"为什么"（洞见），Forked Agent 补的是"发生了什么"（事实）。两者合并形成完整记忆。

5.4 记忆新鲜度四级警告

年龄	行为
0 天（今日）	无特殊处理
1 天（昨日）	无特殊处理
2-29 天	附加"这条记忆是 N 天前的"警告
30 天+	附加"请验证后再引用" + 强调记忆是时间点快照

关键设计：模型对 ISO 时间戳的算术推理能力差。给出"47 天前"比给出"2026-02-12T..."更能触发模型对记忆过时的警觉。

5.5 三级作用域隔离

```
scope = 'user'    → ~/.claude/agent-memory/<agentType>/     跨项目全局
scope = 'project' → <cwd>/.claude/agent-memory/<agentType>/  团队共享
scope = 'local'   → <cwd>/.claude/agent-memory-local/<agentType>/ 仅本地
```

六、工具系统：Fail-Closed 的最小安全窗口

6.1 扁平化内聚接口

Claude Code 的 `Tool` 接口（约 40 个属性）刻意选择**扁平化内聚**而非继承层次。核心驱动：**让 LLM 能一次性"看到"工具的完整行为描述**。

当工具行为被分散在多个 mixin 或继承层级中，LLM 很难准确推断出全部能力边界。通过将"工具如何看待自己"的全部属性集中在同一接口，每个工具的自我描述是完整且内聚的。

6.2 默认值的双重哲学

```
const TOOL_DEFAULTS = {
  isEnabled: () => true,            // 默认启用（而非默认禁用）
  isConcurrencySafe: () => false,   // 默认不并发（保守）
  isReadOnly: () => false,          // 默认有副作用（安全导向）
}
```

- `isEnabled: true`：默认状态优先，禁用需要显式声明
- `isConcurrencySafe: false`：**真正的 fail-closed，不确认安全就保守**

6.3 StreamingToolExecutor 的并发哲学

并发模型不是"最大并行度"模式，而是**最小安全窗口**模式：只要队列中存在任何非并发安全工具，后续所有工具都必须串行等待。

这承认了一个事实：Bash 命令之间存在隐式依赖（环境变量、目录切换）。`[Read, Bash, Read]` 调用时，Read 并发执行，Bash 阻断后续一切。

6.4 八阶段工具执行管线

```
入参解析（Zod）
    │
自定义校验 ──────────────────────┐
    │                            │ 推测性分类器（并行启动）
PostToolUse Hooks ──────────────┤
    │                            │
权限决策链（Hook → 规则 → 模式）      │
    │                            │
tool.call() 执行                  │
    │                            │
PostToolUse Hooks ──────────────┘
    │
错误处理（分层分类）
```

Phase 3 推测性执行是性能亮点：Bash 命令的分类器检查在权限检查和 Hook 执行**并行启动**，用户感知延迟显著降低。

6.5 三级中断体系

```
用户 ESC ──→ abortController（终止整个 turn）
Bash 错误 ──→ siblingAbortController（仅终止同级 bash 进程）
权限拒绝 ──→ toolAbortController（上报给 query loop）
```

`siblingAbortController` 是 `toolUseContext.abortController` 的子级，触发时终止其他并发的 Bash 子进程，但**不触发 ESC 中断**——精准隔离，不扩大打击面。

七、Subagent 系统：隔离与缓存的双重优化

7.1 Fork Subagent 的缓存优化原理

Fork 是 Claude Code 最精妙的设计之一。通过 `buildForkedMessages` 构建与父 API 请求**字节级一致**的消息序列：

1. 深拷贝父的 assistant 消息（含 tool_use blocks）
2. 为每个 tool_use 创建**硬编码常量**的占位 tool_result（`FORK_PLACEHOLDER_RESULT`）
3. 组装为 `[cloned_assistant, ...placeholders, directive]`

关键：占位文本是硬编码常量，序列化结果完全相同 → API cache key 匹配 → 直接返回 `cache: true`，`response_prefix_bytes_reused: ~90%`。

7.2 递归深度守卫

Fork Subagent 通过注入唯一的 `<FORK_BOILERPLATE_TAG>` XML 标记来防止无限递归嵌套。当 `AgentTool.call()` 检测到消息历史中已存在该标记时，直接拒绝 fork。

7.3 六层工具过滤算法

```
Layer 1: 全局禁止列表（ALL_AGENT_DISALLOWED_TOOLS）
Layer 2: 异步 Agent 白名单（ASYNC_AGENT_ALLOWED_TOOLS）
Layer 3: 自定义 Agent 额外禁止
Layer 4: Agent 定义允许列表（通配符展开）
Layer 5: Agent 定义禁止列表（优先级最高）
Layer 6: 权限模式调整（bypassPermissions / acceptEdits / plan）
```

7.4 三种 Agent 启动模式对比

模式	触发条件	API Cache 命中率
Fresh Subagent	显式 `subagent_type`	低（每次新 prefix）
Fork Subagent	`forkEnabled=true` + 无 type	**高（~90% prefix 复用）**
Remote Agent	`isolation: 'remote'`	取决于远程状态

八、查询可靠性：状态机驱动的容错设计

8.1 为什么是状态机

`query.ts` 将整个查询循环包裹在 `while(true)` 状态机中，而非顺序函数链。三条核心驱动：

非确定性：API 调用可能因上下文超限、速率限制、模型过载、用户中断等原因失败，这些无法在调用前预知，必须运行时逐个应对。

协作式并发：`async generator` 的 `yield` 机制使控制流在调用方和 query 循环之间实时切换，流式输出过程中即可开始执行工具。

不可变状态转移：每次循环通过 `state = { ...nextState }` 替换整个状态对象，遗漏字段等于遗漏代码，在 code review 中一目了然。

8.2 防御式三层降级

第一层（Withhold）：可恢复错误在流式循环中捕获但不暴露给调用方。原因：SDK 消费者可能在看到 `error` 字段时终止会话，使恢复逻辑永远无法执行。

第二层（Recover）：turn 结束后尝试自动恢复。

- PTL（Prompt Too Long）：`pre-call` 阻止 → `context collapse drain` → `reactive compact`
- MOT（Max Output Tokens）：`8k→64k` 升级 → 多轮恢复消息注入
- 529（Overload）：重试×3 → 模型降级 → 暴露错误

第三层（Expose）：所有恢复路径耗尽后暴露错误，调用 `executeStopFailureHooks`（而非完整 stop hooks），防止死亡螺旋。

8.3 529 防雪崩设计

529 错误具有独特的网络效应：每次重试加重 API 网关负担，形成正反馈循环。Claude Code 的设计：**只有前台用户等待的 4 类查询（`repl_main_thread`、`sdk`、`compact`、`verification_agent`）才重试 529；所有后台任务立即失败绝不重试。**

8.4 递减回报检测

连续 3 轮 token 增量均 < 500 时保守停止。这是承认了一个现实：Agent 工作产出不是线性随 token 消耗的，前几个 chunk 通常解决了大部分问题（"easy wins"），后续主要做边缘处理和重复验证。

8.5 墓碑机制（Tombstoning）

当 streaming fallback 发生时，正在执行的工具结果不是直接删除，而是用 `<tool_use_error>` 墓碑消息占位。这保持了 tool_use 位置索引的完整性，防止后续 tool_result 错位。

九、Hook 系统：生命周期注入的分层安全

9.1 事件 × 类型正交设计

事件（WHEN）：框架在主循环中埋了 22 个检查点，定义"挂在哪里"。

类型（WHAT）：Command / Prompt / Agent / HTTP / Callback，定义"挂什么逻辑"。

同一事件可挂多种类型，同一类型可出现在不同事件——形成灵活网格而非层级结构。新增事件或新增 hook 类型都不需要修改另一方代码。

9.2 七层纵深安全防御

```
L1: 工作区信任检查（workspace trust）
L2: ManagedOnly 策略（企业托管环境）
L3: 全局禁用开关
L4: HTTP URL 白名单
L5: SSRF DNS 保护（阻止私有 IP 请求）
L6: Header 注入防护（清理 CR/LF/NUL）
L7: 环境变量白名单（不在白名单的 $VAR 替换为空）
```

关键设计：信任检查在 `executeHooks()` 函数入口集中进行，而非分散在每个 hook 执行器中。这确保每种 hook 类型都自动继承安全检查。

9.3 快照隔离配置

Hook 配置在**应用启动时**拍摄快照，运行时直接读取内存。解决三个问题：性能（一级检查点如 PostToolUse 高频触发）、一致性（防止外部编辑导致同一会话行为不一致）、安全性（快照在信任对话框显示前捕获）。

9.4 失败宽容，结果严格

Hook 自身执行错误（超时、解析失败）→ `non_blocking_error`，不阻止主流程。但返回的决策结果（`continue: false`、`stopReason`）严格处理。

十、多 Agent 协作：协调者模式

10.1 中心化星型拓扑

选择中心化而非去中心化的三个约束：

1. **权限审批必须集中**：去中心化无法保证一致的安全边界
2. **上下文窗口有限**：去中心化意味着每个 Agent 都要维护自己的上下文副本
3. **文件系统 IPC 天然适合星型拓扑**：所有消息经过领导

领导定位：Coordinator Agent 的系统提示词开篇明义——"PLAN, ASSIGN, and SYNTHESIZE — not to execute directly." 协调者不亲自执行任务，只负责任务规划、子 Agent 调度、结果综合。

10.2 Spawn vs Continue 决策矩阵

```
USE SPAWN WHEN:
  - Task requires different tool sets (search vs edit)
  - Independent subtasks that can run in parallel
  - Specialized expertise (frontend, backend, infra)

USE CONTINUE WHEN:
  - Next step logically depends on current step's result
  - Context overlap > 60% with current task
  - Subtask is a natural continuation
```

10.3 双层 AbortController 的 UX 哲学

- **生命周期控制器**：控制 teammate 的生死，代表 Team Lead 的"死亡权"
- **工作级别控制器**：Escape 键只停止当前工作，代表用户的"暂停权"

用户按 Escape 时，teammate 进入 idle 状态但不消失——Team Lead 之前发给它的消息仍然有效。

10.4 preserveMode 防提权

Worker 获得的权限决策历史不会"溢出"到 Team Lead 的全局状态，防止 Team Lead 批准了 Worker 的某次操作后，污染 Leader 自己的权限上下文。

十一、十三层上下文动态注入管道

Claude Code 的上下文注入不是单一机制，而是 **13 个各司其职的子系统**，各层职责清晰：

层	名称	注入内容	时机
1	System Reminder	模式切换、系统状态、工具结果	随消息注入
2	CLAUDE.md / 记忆文件	项目规则、用户偏好	SessionStart
3	Auto Memory	自动持久化记忆	每轮查询前
4	Session Memory	会话级临时状态	会话内跨轮
5	ExtractMemories	会话末记忆提取（Forked Agent）	SessionEnd
6	Hook 系统	生命周期注入	各检查点
7	System Prompt 分段	动态边界标记、缓存断裂检测	API 调用前
8	条件化规则	基于文件路径的规则激活	动态
9	Post-Sampling Hook	采样后注入（Beta）	模型输出后
10	appendSystemPrompt	额外注入（最低优先级）	任意时机
11	Coordinator Prompt	协调者模式专用提示	Coordinator 模式
12	Compact 边界	压缩触发后的标记	压缩后
13	Skill 上下文	Skill 描述、工具列表	Skill 调用时

分层设计的关键洞察：每层都是**可独立启用/禁用/配置的**，通过 feature flag 控制，而非硬编码的 if-else 链。上下文管理的演进是模块化的。

十二、关键设计哲学

哲学一：渐进式复杂性纪律

每一个"为什么不用 X"的答案都是对**工程实用性**的坚持：

场景	选择	原因
用户输入分类	O(1) 字符串规则	LLM 分类成本高，CLI 输入有强结构性
并发安全分区	`isConcurrencySafe` 静态判断	工具语义可提前分类，动态分析开销大
缓存策略	重复加载发生再加 memoize	无重复场景下缓存是纯粹开销
异步后台	GrowthBook 灰度验证后才全量	不稳定的后台行为会损害用户体验

哲学二：防御式分层（Defense in Depth）

每种故障至少有两层降级路径，每层只处理自己擅长的问题：

- 压缩：snip → microcompact → contextCollapse → autocompact
- 工具执行：并发安全分区 + 三级中断
- 错误处理：暂挂 → 恢复 → 暴露
- Hook 安全：信任检查 → 快照隔离 → 白名单 → SSRF 防护

哲学三：不可变状态转移

QueryLoop 中 9 个 `continue` 跳转点全部通过替换整个 State 对象而非修改字段实现。状态一致性得到结构性保证——遗漏字段等于遗漏代码，在 code review 中一目了然。

哲学四：API 缓存命中是最核心的性能目标

几乎所有复杂机制都围绕这一目标设计：

- Fork Subagent 的字节级消息克隆 → 保护父会话缓存 key
- Microcompact 的 `cache_edits` → 清理上下文但不破坏缓存
- Side Query 的独立调用 → 利用主会话缓存近乎免费
- 工具列表按字母序排列 → 确保序列化一致性

哲学五：可观测即架构

关键业务决策点（autocompact 触发、模型降级、token budget 停止、529 过载）都同步输出结构化的埋点数据。**产品逻辑和埋点逻辑是同一套代码、同一个 PR、同一个 Review 完成。**

十三、核心工程洞见总结

洞见一：KV 缓存是性能的基石

Fork Subagent 的字节克隆、Microcompact 的 `cache_edits`、Side Query 的独立调用——所有复杂机制最终都围绕保护或利用 API Prompt Cache。每一次 KV 缓存未命中都是直接的货币损耗。

洞见二：渐进式复杂性是最核心的工程纪律

规则够用就不上 LLM，线性扫描够用就不做最优分配，缓存复用确实发生再加 memoize。在每个决策点选择当前规模下最简单的方案，不过度设计，不过早优化。

洞见三：失败是设计的第一公民

死亡螺旋防护、熔断器、529 防雪崩、递减回报检测——系统被设计为在最坏情况下也能优雅降级。每种错误都有明确的失败模式和恢复路径，而非让系统崩溃或无限重试。

洞见四：分层优于统一

所有复杂问题都避免"大一统"方案——上下文注入 13 层、压缩 4 层、工具执行并发分区、Agent 6 种类型。每层职责边界清晰、触发条件明确，通过标准接口而非直接调用串联。

洞见五：隔离即安全

工具隔离、文件系统隔离（Worktree）、上下文隔离（Fork）、API 成本隔离（优先级队列）。通过精细的分层控制，让专业 Agent 干专业事，同时不污染主会话状态。